

PROJET CLOUD CHIFFRÉ RSA

Documentation Complète — Version Améliorée

Chiffrement RSA pur par blocs · Hybride RSA+AES-GCM · Authentification Multi-facteurs · Gestion utilisateurs · Quotas · API REST · Interface Web Moderne

Ce document présente l'architecture complète d'un système de stockage cloud simulé avec chiffrement RSA pur. Chaque fichier est découpé en blocs et chiffré individuellement avec la clé publique RSA de l'utilisateur. Ce document couvre les tâches détaillées, les approches possibles, la meilleure approche retenue, les outils recommandés, les pistes d'amélioration, **les nouvelles mesures de sécurité ajoutées**, et **les étapes complètes de création du frontend**.

MODULE	CONTENU	PAGES
1	Infrastructure & Base de données	2–3
2	Moteur RSA pur	4–5
3	Authentification & Gestion utilisateurs	6–7
4	Gestion des fichiers & Quotas	8–9
5	API REST Backend	10
6	Interface utilisateur (Frontend)	11
7 ★ NOUVEAU	Mesures de sécurité avancées	12–14
8 ★ NOUVEAU	Étapes création Frontend moderne	15–17
—	Comparaison des approches	18
—	Meilleurs outils par tâche	19
—	Améliorations possibles	20
—	Ordre de réalisation	21

MODULE 1 — Infrastructure & Base de données

1.1 Initialisation du projet

- Créer la structure de dossiers : /backend, /frontend, /storage, /tests
- Configurer l'environnement virtuel Python (venv)
- Rédiger le fichier requirements.txt avec toutes les dépendances
- Initialiser un dépôt Git avec .gitignore adapté (ne jamais versionner les clés privées)

Outils : Python venv | Git | VS Code

1.2 Conception du schéma de base de données

- Table users : id, email, password_hash, public_key (PEM), quota_total (octets), quota_used, created_at
- Table files : id, user_id (FK), original_name, stored_uuid, size_bytes, upload_date
- Table logs (optionnel) : id, user_id, action, ip_address, timestamp
- Définir les contraintes d'intégrité (UNIQUE email, FK avec CASCADE DELETE)

Outils : SQLite (dev) | PostgreSQL (prod) | DB Browser for SQLite

1.3 Initialisation & migration de la base

- Script init_db.py qui crée les tables si elles n'existent pas
- Script reset_db.py pour vider la base lors des tests
- Vérification de la connexion au démarrage de l'API

Outils : SQLAlchemy ORM | Alembic (migrations) | sqlite3 (stdlib)

1.4 Organisation du stockage physique

- Dossier racine /storage/ créé automatiquement si absent
- À l'inscription : création automatique de /storage/{user_id}/
- Les fichiers sont stockés sous un nom UUID aléatoire (ex: a3f9c2d1.enc)
- Jamais le nom original en clair sur le disque

Outils : os / pathlib (stdlib) | uuid (stdlib)

MODULE 2 — Moteur RSA pur (cœur du projet)

RSA ne peut chiffrer qu'un volume limité de données par opération (245 octets pour une clé 2048 bits avec OAEP). La solution est de découper le fichier en blocs, chiffrer chaque bloc individuellement, puis concaténer les blocs chiffrés. Chaque bloc chiffré fait exactement 256 octets.

Étape	Opération	Taille
Fichier original	Lu en binaire brut	N octets
Découpage	Blocs de 190 octets (OAEP 2048 bits)	ceil(N/190) blocs
Chiffrement	RSA-OAEP par bloc + clé publique	256 octets/bloc
Stockage	Blocs concaténés → fichier .enc	~N × 1.35
Déchiffrement	Blocs de 256 octets → RSA-OAEP + clé privée	Fichier original

2.1 Génération des clés RSA

- Générer une paire clé publique / clé privée RSA 2048 bits minimum
- Sérialiser en format PEM (texte encodé Base64) pour stockage et transport
- Fonction `load_public_key(pem)` et `load_private_key(pem)` pour désérialisation
- La clé publique est stockée en base de données. La clé privée est retournée une seule fois à l'utilisateur

Outils : `PyCryptodome (RSA.generate)` | `cryptography (rsa.generate_private_key)`

2.2 Chiffrement d'un fichier par blocs

- Lire le fichier en mode binaire
- Calculer la taille de bloc : 190 octets pour RSA-2048 avec OAEP
- Itérer sur le fichier par tranches de 190 octets
- Chiffrer chaque tranche : `PKCS1_OAEP.encrypt(bloc)`
- Concaténer tous les blocs chiffrés (chacun fait 256 octets) et écrire le fichier .enc

Outils : `PyCryptodome PKCS1_OAEP` | `io.BytesIO`

2.3 Déchiffrement d'un fichier par blocs

- Lire le fichier .enc en mode binaire
- Découper en blocs de 256 octets exactement (taille fixe après chiffrement RSA-2048)
- Déchiffrer chaque bloc : `PKCS1_OAEP.decrypt(bloc)`
- Concaténer les blocs déchiffrés et reconstituer le fichier original
- Supprimer le fichier temporaire déchiffré après envoi

Outils : `PyCryptodome PKCS1_OAEP`

2.4 Tests unitaires du moteur RSA

- Test chiffrement → déchiffrement sur fichier texte simple
- Test sur fichier binaire (image PNG, PDF)
- Test fichier vide (0 octets) et fichier d'un seul octet
- Vérifier l'identité du fichier reconstruit via hash SHA-256
- Mesurer le temps de traitement selon la taille du fichier

Outils : `pytest` | `hashlib (stdlib)`

MODULE 3 — Authentification & Gestion utilisateurs

3.1 Inscription

- Recevoir email + mot de passe via POST /auth/register
- Valider format email (regex) et force du mot de passe (8 cars min, chiffre, majuscule)
- Hasher le mot de passe avec bcrypt (coût 12)
- Appeler le module RSA pour générer la paire de clés
- Stocker clé publique en base. Retourner clé privée une seule fois au client
- Créer automatiquement le dossier /storage/{user_id}/
- Attribuer un quota par défaut (ex : 500 Mo)

Outils : `bcrypt` | `email-validator` | `PyCryptodome`

3.2 Connexion

- Recevoir email + mot de passe via POST /auth/login
- Vérifier le hash bcrypt stocké en base
- Générer un token JWT signé (expiration 24h)
- Gérer les erreurs : utilisateur inexistant (401), mauvais mot de passe (401)
- Ne jamais préciser si c'est l'email ou le mot de passe qui est incorrect (sécurité)

Outils : `python-jose (JWT)` | `bcrypt`

3.3 Middleware d'authentification

- Décorateur FastAPI Depends() appliqué à toutes les routes protégées
- Extraire le token JWT du header Authorization: Bearer
- Vérifier signature et expiration du JWT
- Injecter l'ID utilisateur dans le contexte de la requête
- Retourner HTTP 401 si token invalide ou expiré

Outils : `FastAPI Depends` | `python-jose`

3.4 Gestion du profil

- GET /user/profile → email, quota total, quota utilisé, date d'inscription
- PUT /user/password → vérifier ancien mot de passe, hasher le nouveau
- DELETE /user/account → supprimer tous les fichiers physiques puis l'entrée en base

Outils : `FastAPI` | `SQLAlchemy`

MODULE 4 — Gestion des fichiers & Quotas

4.1 Vérification du quota

- Fonction `check_quota(user_id, file_size) → bool`
- Requête : `SELECT quota_used, quota_total FROM users WHERE id = ?`
- Condition : `quota_used + file_size <= quota_total`
- Retourner aussi l'espace restant pour affichage dans l'erreur

Outils : SQLAlchemy | Python stdlib

4.2 Téléversement (Upload)

- Recevoir le fichier via multipart/form-data (POST /files/upload)
- Vérifier le quota AVANT de traiter le fichier
- Si quota dépassé : HTTP 507 Insufficient Storage + message détaillé
- Chiffrer le fichier avec le moteur RSA (clé publique de l'utilisateur en base)
- Générer un UUID comme nom de stockage → sauvegarder `/storage/{user_id}/{uuid}.enc`
- Mettre à jour `quota_used` en base de données
- Enregistrer les métadonnées dans la table files

Outils : FastAPI UploadFile | uuid | PyCryptodome

4.3 Téléchargement (Download)

- POST /files/download/{file_id} — POST car on envoie la clé privée dans le body
- Vérifier que `files.user_id == utilisateur connecté` (pas d'accès cross-user)
- Recevoir la clé privée PEM dans le body JSON
- Lire le fichier .enc depuis `/storage/{user_id}/{uuid}.enc`
- Déchiffrer avec le moteur RSA + clé privée reçue
- Retourner le fichier avec son nom original et Content-Disposition
- Ne jamais stocker la clé privée reçue

Outils : FastAPI StreamingResponse | PyCryptodome

4.4 Liste & suppression des fichiers

- GET /files/ → liste des fichiers : nom original, taille (Mo), date d'upload
- Ne jamais exposer l'UUID de stockage dans la réponse
- DELETE /files/{file_id} → vérifier propriété, supprimer fichier physique + BDD
- Décrémenter `quota_used` après suppression

Outils : FastAPI | os.remove | SQLAlchemy

MODULE 5 — API REST Backend

Méthode	Route	Description	Auth
POST	/auth/register	Inscription + génération clés RSA	Non
POST	/auth/login	Connexion + retour JWT	Non
POST	/auth/refresh	Renouveler le JWT via refresh token	Non
POST	/auth/logout	Invalider le token (blacklist)	Oui
GET	/user/profile	Profil + quota utilisé/total	Oui
PUT	/user/password	Changement de mot de passe	Oui
DELETE	/user/account	Suppression du compte	Oui
POST	/files/upload	Upload + chiffrement RSA	Oui
GET	/files/	Liste des fichiers de l'utilisateur	Oui
POST	/files/download/{id}	Download + déchiffrement RSA	Oui
DELETE	/files/{id}	Suppression d'un fichier	Oui
GET	/health	Vérification que l'API est en ligne	Non

5.1 Configuration FastAPI

- Configurer CORS pour autoriser le frontend (origines, méthodes, headers)
- Handler global d'exceptions pour retourner des erreurs JSON propres
- Middleware de logging des requêtes
- Endpoint /health pour vérifier que l'API est en ligne

Outils : FastAPI | uvicorn | python-multipart

5.2 Documentation & Tests API

- Swagger UI auto-généré par FastAPI : accessible sur /docs
- Tester chaque endpoint avec Postman ou HTTPie
- Vérifier les codes HTTP : 200, 201, 400, 401, 403, 404, 507
- Tests d'intégration : inscription → connexion → upload → download → suppression

Outils : FastAPI /docs | Postman | pytest + httpx

MODULE 6 — Interface utilisateur (Frontend) — Existant

6.1 Page d'inscription

- Formulaire email + mot de passe avec validation en temps réel
- Après inscription : afficher la clé privée RSA générée
- Bouton 'Télécharger ma clé privée' → fichier .pem local
- Message d'avertissement fort : cette clé ne sera JAMAIS ré-affichée
- Redirection automatique vers le dashboard après téléchargement confirmé

Outils : React | Tailwind CSS | Axios

6.2 Page de connexion

- Formulaire email + mot de passe
- Stocker le JWT en mémoire (variable React, pas localStorage)
- Gérer l'expiration : rediriger vers login si 401 reçu

Outils : React useState | Axios interceptors

6.3 Dashboard principal

- Barre de quota visuelle : X Mo utilisés sur Y Mo (barre colorée)
- Barre rouge si > 90% du quota
- Liste des fichiers : nom, taille, date, boutons Télécharger / Supprimer
- Rafraîchissement automatique après chaque action

Outils : React | Tailwind CSS | recharts (barre quota)

6.4 Composant Upload

- Drag & drop ou sélecteur de fichier natif
- Affichage de la taille avant envoi + estimation si quota sera dépassé
- Barre de progression pendant l'upload
- Message de succès ou erreur quota clair

Outils : React | FormData API | Axios onUploadProgress

6.5 Composant Download

- Champ pour charger ou coller la clé privée RSA (.pem)
- Envoi de la clé + ID fichier au backend pour déchiffrement
- Déclenchement du téléchargement du fichier reconstruit via Blob
- Vider la clé privée de la mémoire après utilisation

Outils : React | Blob / URL.createObjectURL | Axios

MODULE 7 ★ NOUVEAU — Mesures de Sécurité Avancées

Ce module détaille toutes les mesures de sécurité à ajouter au projet de base pour atteindre un niveau proche des standards professionnels.

7.1 Authentification renforcée

Refresh Tokens (manquant dans la version de base)

- Le JWT actuel expire en 24h sans possibilité de renouvellement propre
- Ajouter un access token court (15–30 min) + un refresh token (7–30 jours) stocké en base
- Route POST /auth/refresh : vérifie le refresh token en base → retourne un nouvel access token
- Révoquer le refresh token à la déconnexion ou au changement de mot de passe
- Outil : python-jose + table refresh_tokens en base (token_hash, user_id, expires_at, revoked)

Protection Brute Force

- Rate limiting sur /auth/login : maximum 5 tentatives par 10 minutes par IP
- Après 10 échecs consécutifs : blocage temporaire du compte (30 min) + email d'alerte
- Compteur de tentatives stocké dans Redis avec TTL automatique
- Outil : slowapi (rate limiter FastAPI) + Redis

2FA TOTP — Authentification à deux facteurs

- À l'activation du 2FA : générer un secret TOTP unique par utilisateur
- Afficher un QR code à scanner avec Google Authenticator ou Authy
- À la connexion : après email/mdp valides, demander le code à 6 chiffres
- Stocker le secret TOTP chiffré en base (jamais en clair)
- Prévoir des codes de récupération à usage unique en cas de perte du téléphone
- Outil : pyotp | qrcode | Pillow

Invalidation des tokens JWT (faille critique)

- Un JWT reste valide même après déconnexion ou compromission — faille majeure
- Solution A (tokens opaques) : stocker UUID en base, vérifier à chaque requête
- Solution B (blacklist Redis) : à la déconnexion, ajouter le JTI du token dans Redis avec TTL = expiration
- Le middleware vérifie que le JTI n'est pas dans la blacklist avant d'accepter la requête
- Outil : Redis (TTL automatique) + champ jti dans le payload JWT

Vérification de l'email à l'inscription

- Envoyer un lien de confirmation (token UUID, TTL 24h) avant d'activer le compte
- Compte en état pending jusqu'à confirmation — bloque les comptes faux
- Route GET /auth/verify/{token} → active le compte et invalide le token
- Outil : FastMail / SMTP + table email_verifications en base

7.2 Chiffrement renforcé — RSA + AES-GCM

■ **Problème fondamental du RSA pur : RSA bloc par bloc est lent et ne garantit pas l'intégrité des données. Un attaquant peut modifier les blocs chiffrés sans être détecté. AES-GCM résout les deux problèmes.**

Architecture Hybride RSA + AES-256-GCM (recommandée)

Étape	Opération	Détail
1	Générer une clé AES-256 aléatoire	32 octets par fichier, jamais réutilisée
2	Chiffrer le fichier avec AES-256-GCM	Rapide + tag d'authentification intégré

3	Chiffrer la clé AES avec RSA-OAEP	Seulement 32 octets passent par RSA
4	Stocker : clé AES chiffrée + nonce + tag + fichier chiffré	Format .enc structuré
5	Déchiffrement : RSA déchiffre la clé AES → AES-GCM déchiffre le fichier	

Pourquoi AES-GCM ?

- AES-GCM inclut un tag d'authentification (MAC) de 16 octets — détecte toute modification du fichier chiffré
- RSA seul n'authentifie pas — un attaquant peut altérer les blocs sans que tu t'en rendes compte
- AES-GCM est 100× plus rapide que RSA pur sur les gros fichiers
- Nonce de 96 bits unique par fichier, stocké en clair avec le fichier chiffré
- Outil : cryptography — AESGCM de cryptography.hazmat.primitives.ciphers.aead

Protection de la clé privée par mot de passe (Argon2)

- La clé privée .pem retournée en clair est vulnérable si le fichier est volé
- Dériver une clé AES-256 depuis le mot de passe utilisateur avec Argon2id
- Chiffrer le PEM de la clé privée avec cette clé dérivée avant de le retourner
- L'utilisateur télécharge un .pem chiffré — inutilisable sans le mot de passe
- Pour déchiffrer un fichier : entrer le mot de passe → dériver la clé → déchiffrer le PEM → déchiffrer le fichier
- Outil : argon2-cffi | cryptography

Signature numérique RSA-PSS (intégrité des fichiers)

- Signer le hash SHA-256 du fichier original avec la clé privée à l'upload
- Stocker la signature avec les métadonnées du fichier en base
- À chaque accès : vérifier la signature pour détecter toute altération du fichier chiffré
- Garantit l'authenticité (seul le propriétaire pouvait signer) et l'intégrité
- Outil : PyCryptodome — pss.sign / pss.verify

7.3 Base de données & stockage sécurisés

- Chiffrer original_name en base avec AES-256 (clé serveur via variable d'environnement) — évite la fuite de métadonnées
- Sous-dossiers hashés : /storage/{sha256(user_id)}/{uuid}.enc — rend l'énumération impossible
- Paramétrer toutes les requêtes SQL (pas de concaténation directe) — déjà garanti par SQLAlchemy ORM
- Chiffrement de la base SQLite au repos avec SQLCipher (dev) ou chiffrement disque PostgreSQL (prod)

7.4 Transport & API sécurisés

- HTTPS obligatoire en production avec redirection 301 du HTTP vers HTTPS
- Header Strict-Transport-Security (HSTS) : max-age=31536000; includeSubDomains
- Header Cache-Control: no-store sur les réponses de download (pas de mise en cache navigateur)
- Content-Security-Policy strict pour le frontend (pas d'injection XSS)
- Header X-Content-Type-Options: nosniff pour éviter le MIME sniffing

Tableau de priorité des mesures de sécurité

Priorité	Mesure	Algo / Outil
■ Critique	Hybride RSA + AES-GCM	cryptography — AESGCM
■ Critique	Clé privée protégée par mot de passe	Argon2id + AES-256
■ Haute	Refresh tokens + blacklist JWT	Redis + python-jose
■ Haute	Rate limiting login (brute force)	slowapi + Redis

■ Moyenne	Signature RSA-PSS des fichiers	PyCryptodome PSS
■ Moyenne	2FA TOTP	pyotp + qrcode
■ Moyenne	Invalidation JWT à la déconnexion	Redis JTI blacklist
■ Optionnel	Chiffrement métadonnées en base	AES-256 clé serveur
■ Optionnel	Vérification email inscription	SMTP + token UUID
■ Optionnel	HTTPS + HSTS + CSP headers	nginx / FastAPI middleware

MODULE 8 ★ NOUVEAU — Étapes Création Frontend Moderne

Ce module détaille la création d'un frontend attrayant, animé et fonctionnel, avec gestion de session, modification de compte, et trois modes d'authentification : email/mot de passe, empreinte digitale (Web Authentication API) et Face ID.

8.1 Stack technique & initialisation

- Vite + React 18 + TypeScript pour un projet moderne et typé
- Tailwind CSS v3 pour le styling utilitaire (pas de CSS custom sauf animations)
- Framer Motion pour les animations fluides (transitions, hover, apparition)
- Zustand pour la gestion d'état global (auth, fichiers, quota)
- Axios avec interceptors pour la gestion automatique des tokens
- React Router v6 pour la navigation protégée (PrivateRoute)
- React Hot Toast pour les notifications non intrusives

```
Commandes : npm create vite@latest cloud-rsa-frontend -- --template react-ts && npm install
tailwindcss framer-motion zustand axios react-router-dom react-hot-toast
```

8.2 Design System — Thème & Animations

- Palette sombre : background #0F1923, cards #1A2535, accent cyan #00D4FF, violet #7B61FF
- Typographie : Inter (Google Fonts) — chargée via @fontsource/inter
- Particules d'arrière-plan animées (react-particles) sur les pages auth pour l'effet 'tech'
- Glassmorphism : cards avec backdrop-blur-md et border transparent semi-opaque
- Animations Framer Motion : fadeIn à l'apparition, slideUp pour les listes, pulse sur les erreurs
- Micro-interactions : hover scale 1.02 sur les boutons, ripple effect au clic
- Barre de quota avec gradient animé (vert → orange → rouge selon le pourcentage)

8.3 Page d'authentification — 3 modes

Mode 1 — Email + Mot de passe (classique)

- Formulaire avec validation en temps réel (Zod ou React Hook Form)
- Indicateur de force du mot de passe (barre colorée progressant avec la complexité)
- Show/hide password avec icône toggle
- Bouton de connexion avec spinner pendant la requête

Mode 2 — Empreinte digitale (WebAuthn / FIDO2)

- API : navigator.credentials.create() à l'inscription, navigator.credentials.get() à la connexion
- Flux d'inscription : serveur génère un challenge → navigateur appelle l'authenticator → envoie la réponse signée au serveur
- Flux de connexion : serveur envoie un challenge → navigateur vérifie l'empreinte → retourne une assertion signée
- Backend : stocker la clé publique WebAuthn en base (table webauthn_credentials)
- Librairie frontend : @simplewebauthn/browser | Backend : py_webauthn (Python)
- Afficher le bouton uniquement si window.PublicKeyCredential est disponible dans le navigateur
- UX : icône empreinte digitale animée, feedback 'Placez votre doigt...' pendant la lecture

Mode 3 — Face ID (WebAuthn avec caméra / plateforme)

- Face ID utilise aussi WebAuthn — le même flux que l'empreinte digitale
- Sur iOS Safari et macOS : l'authenticateur plateforme gère automatiquement Face ID vs Touch ID
- Paramètre clé à l'inscription : authenticatorAttachment: 'platform' force l'authenticateur intégré
- Le navigateur choisit Face ID ou empreinte selon le device — pas besoin de différencier côté code

- Sur Android : Google Password Manager ou empreinte selon le device
- Afficher les icônes adaptatives : détecter iOS/macOS (Face ID) vs autres (empreinte)

Note : WebAuthn fonctionne en HTTPS uniquement. En développement local, utiliser localhost (exception navigateur) ou un tunnel HTTPS (ngrok).

8.4 Restauration de session (token expiré)

- À chaque démarrage de l'app : vérifier si un refresh token existe (stocké en cookie httpOnly sécurisé)
- Si access token expiré (401) : Axios interceptor appelle automatiquement POST /auth/refresh
- Si refresh réussi : retenter la requête originale avec le nouveau token — transparent pour l'utilisateur
- Si refresh échoué (refresh token expiré ou révoqué) : redirection vers /login avec message 'Session expirée'
- Zustand store : état isRefreshing pour éviter les appels parallèles de refresh
- Cookie httpOnly pour le refresh token : inaccessible au JavaScript, protégé contre XSS

Code — Axios interceptor de refresh automatique :

```
axiosInstance.interceptors.response.use(null, async (error) => {
  if (error.response?.status === 401 && !error.config._retry) {
    error.config._retry = true;
    await authStore.refresh(); // appelle POST /auth/refresh
    return axiosInstance(error.config); // retente la requête
  }
  return Promise.reject(error);
});
```

8.5 Page de modification du compte

- Section 'Informations' : modifier l'email (confirmation par lien envoyé au nouvel email)
- Section 'Sécurité' : changer le mot de passe (ancien + nouveau + confirmation)
- Section 'Authentification biométrique' : activer/désactiver empreinte ou Face ID
- Section 'Double authentification' : activer/désactiver TOTP avec affichage du QR code
- Section 'Clés RSA' : afficher la date de génération, bouton pour régénérer une nouvelle paire
- Section 'Danger zone' : supprimer le compte (confirmation par saisie de l'email + mdp)
- Toutes les actions sensibles demandent une re-confirmation du mot de passe actuel

8.6 Dashboard animé

- Header avec avatar initiales, nom d'utilisateur, et bouton de déconnexion
- Widget quota : cercle de progression animé (SVG stroke-dasharray) + barre linéaire
- Alerte visuelle (banner rouge clignotant) si quota > 90%
- Liste de fichiers avec animations d'entrée en cascade (stagger Framer Motion)
- Colonnes : nom (tronqué avec tooltip), type (icône), taille, date, actions
- Filtres : recherche par nom, tri par date/taille, filtre par type de fichier
- Bouton upload flottant (FAB) avec animation de rotation à l'ouverture de la modale

8.7 Composant Upload amélioré

- Zone drag & drop avec animation de 'glimmer' quand un fichier est survolé
- Prévisualisation du fichier avant upload (icône type + nom + taille)
- Estimation en temps réel : 'Il vous restera X Mo après cet upload'
- Barre de progression avec pourcentage et estimation du temps restant
- File queue : possibilité d'uploader plusieurs fichiers en séquence

8.8 Navigation & Routing protégé

- React Router v6 avec PrivateRoute : redirige vers /login si non authentifié

- Routes : / (landing), /login, /register, /dashboard, /account, /files/{id}
- Transitions de page animées avec Framer Motion AnimatePresence
- Sidebar responsive : menu hamburger sur mobile, sidebar fixe sur desktop
- Breadcrumbs dynamiques pour la navigation dans les dossiers (évolution future)

Résumé des librairies frontend

Librairie	Version	Rôle
React + TypeScript	18 + 5	Framework UI + typage
Vite	5+	Build tool ultra-rapide
Tailwind CSS	3+	Styling utilitaire
Framer Motion	11+	Animations & transitions
Zustand	4+	État global (auth, fichiers)
Axios	1.6+	Requêtes HTTP + interceptors
React Router v6	6+	Navigation + routes protégées
@simplewebauthn/browser	9+	WebAuthn (empreinte / Face ID)
React Hook Form + Zod	7+ / 3+	Formulaires + validation
React Hot Toast	2+	Notifications toast
Framer Motion	11+	Animations particules fond
Lucide React	0.4+	Icônes modernes SVG

COMPARAISON DES APPROCHES

Approche 1 — RSA pur par blocs (approche de base retenue)

Chaque fichier est découpé en blocs de 190 octets, chaque bloc est chiffré individuellement avec RSA-OAEP 2048 bits. Les blocs chiffrés (256 octets chacun) sont concaténés.

Critère	Évaluation	Détail
Sécurité	Bonne (avec OAEP)	OAEP ajoute du padding aléatoire, évite les attaques par pattern
Performance	Limitée	Lente sur gros fichiers (1 op RSA par 190 octets)
Complexité	Moyenne	Gestion manuelle des blocs, logique simple
Pédagogie	Excellente	Expose directement le fonctionnement interne de RSA
Scalabilité	Faible	Fichiers > 10 Mo deviennent très lents
Intégrité	Absente	Pas de vérification d'intégrité intégrée

Approche 2 — Chiffrement hybride RSA + AES-GCM ★ Recommandée

Une clé AES-256 aléatoire chiffre le fichier (rapide, authentifié). La clé AES est chiffrée avec RSA-OAEP. Seuls 32 octets passent par RSA. C'est le standard TLS, PGP, Signal.

Critère	Évaluation	Détail
Sécurité	Excellente	Standard cryptographique mondial + intégrité GCM
Performance	Excellente	RSA sur 32 octets seulement, AES ultra-rapide
Complexité	Légèrement plus	Deux algorithmes, structure de fichier plus riche
Pédagogie	Très bonne	Illustre l'usage réel de RSA en production
Scalabilité	Excellente	Fonctionne sur plusieurs Go sans problème
Intégrité	Native	GCM inclut un tag d'authentification — toute modif détectée

Approche 3 — Chiffrement côté client (navigateur)

Le chiffrement/déchiffrement se fait dans le navigateur avec l'API Web Crypto avant l'upload. Le serveur ne reçoit jamais les données en clair.

Critère	Évaluation	Détail
Sécurité	Maximale	Le serveur ne voit jamais les données en clair
Performance	Moyenne	Web Crypto API est rapide, mais limite le RSA pur
Complexité	Élevée	Gestion des clés entièrement côté frontend
Pédagogie	Complexe	Nécessite maîtrise de l'API Web Crypto JS

MEILLEURS OUTILS PAR TÂCHE

Domaine	Outil	Version	Rôle précis
Chiffrement RSA	PyCryptodome	3.20+	RSA.generate, PKCS1_OAEP encrypt/decrypt, PSS sign/verify
Chiffrement AES	cryptography	42+	AESGCM — chiffrement authentifié AES-256-GCM
Dérivation clé	argon2-cffi	23+	Argon2id — protection clé privée par mot de passe
Framework API	FastAPI	0.111+	Routes, validation Pydantic, doc Swagger auto
Serveur ASGI	Uvicorn	0.29+	Serveur de production pour FastAPI
Base de données	SQLAlchemy	2.0+	ORM Python, compatible SQLite et PostgreSQL
Migrations BDD	Alembic	1.13+	Versioning du schéma de base de données
Hash mots de passe	bcrypt	4.1+	Hash sécurisé des mots de passe (coût 12)
Tokens JWT	python-jose	3.3+	Génération et vérification des tokens JWT
Cache / sessions	Redis	7+	Blacklist JWT, rate limiting, refresh tokens
Rate limiting	slowapi	0.1+	Rate limiter pour FastAPI (brute force protection)
2FA TOTP	pyotp	2.9+	Génération et vérification des codes TOTP
WebAuthn backend	py_webauthn	2+	Empreinte digitale et Face ID côté serveur
Tests backend	pytest + httpx	latest	Tests unitaires et d'intégration de l'API
Frontend framework	React + TypeScript	18+	Interface utilisateur dynamique et typée
Build tool	Vite	5+	Build ultra-rapide, HMR instantané
Style CSS	Tailwind CSS	3+	Classes utilitaires, responsive design
Animations	Framer Motion	11+	Animations fluides, transitions de page
État global	Zustand	4+	Auth store, fichiers, état de l'app
Requêtes HTTP	Axios	1.6+	Appels API, interceptors refresh token
WebAuthn frontend	@simplewebauthn/browser	9+	Empreinte / Face ID côté navigateur
Formulaires	React Hook Form + Zod	7+/3+	Validation typée et performante
Base dev	SQLite	stdlib	Base légère pour le développement local
Base prod	PostgreSQL	16+	Base robuste pour le déploiement final
Versioning	Git + GitHub	latest	Suivi des versions, collaboration

AMÉLIORATIONS POSSIBLES

Améliorations de sécurité

- Rotation des clés RSA : régénérer une paire et re-chiffrer tous les fichiers existants avec la nouvelle clé publique
- Chiffrement de la clé privée : chiffrer le .pem avec PBKDF2 ou Argon2 dérivé du mot de passe utilisateur
- Authentification à deux facteurs TOTP : ajouter Google Authenticator à la connexion
- Signature numérique des fichiers : signer chaque fichier chiffré pour garantir intégrité et authenticité
- Logs d'audit : enregistrer chaque accès (upload, download, échec connexion) avec timestamp et IP

Améliorations de performance

- Traitement asynchrone des gros fichiers : workers Celery + Redis pour chiffrer en arrière-plan
- Chiffrement en streaming : lire et chiffrer bloc par bloc sans charger le fichier en mémoire
- Cache des clés publiques : mettre en cache la clé RSA désérialisée pour éviter de la parser à chaque upload
- Parallélisation du chiffrement : multiprocessing Python pour chiffrer plusieurs blocs simultanément (gain 50–75%)

Améliorations fonctionnelles

- Partage de fichiers : re-chiffrer avec la clé publique du destinataire et envoyer une notification
- Versioning des fichiers : conserver les versions précédentes lors d'un re-upload
- Corbeille et récupération : déplacer en corbeille avec rétention 30 jours avant suppression définitive
- Quotas flexibles et plans : niveaux Basique/Standard/Premium avec alertes email à 80% et 95%
- Prévisualisation des fichiers : miniature chiffrée séparément pour images et PDF
- Application mobile React Native : même API, clé privée dans le keystore sécurisé du téléphone

ORDRE DE RÉALISATION RECOMMANDÉ

Phase	Modules	Durée	Livrable
Phase 1 Fondations	1.1 → 1.4 BDD + Stockage	2–3 j	Projet initialisé, BDD créée, dossiers storage prêts
Phase 2 Moteur RSA	2.1 → 2.4 Clés + Chiffrement	3–4 j	Module crypto autonome, chiffrement/déchiffrement validé par hash SHA-256
Phase 3 Auth	3.1 → 3.4 Inscription + JWT	2–3 j	API d'authentification fonctionnelle, testée sur Postman
Phase 4 Fichiers	4.1 → 4.4 Upload + Download	3–4 j	Flux complet upload+chiffrement et download+déchiffrement opérationnel
Phase 5 Sécurité	7.1 → 7.4 Auth + Crypto avancée	3–4 j	AES-GCM hybride, Argon2, refresh tokens, brute force protection
Phase 6 API complète	5.1 → 5.2 Routes + Docs	1–2 j	API REST documentée, tous endpoints testés avec codes HTTP corrects
Phase 7 Frontend	8.1 → 8.8 UI complète	5–7 j	Interface moderne : auth 3 modes, dashboard animé, upload/download, compte
Phase 8 Tests	Tous modules	2–3 j	Tests unitaires, intégration, sécurité — couverture > 80%

Durée totale estimée : 21 à 30 jours pour un développeur seul à temps plein. En équipe de 2 (backend + frontend en parallèle) : réductible à 12–16 jours.